

sd06. Software Load Balancer (L4 / L7)

First-hand 1p3a P0 (bank #6): "Design a software load balancer, L4 or L7 free to choose; discussion around load-balancing policies / healthchecks." JD tie-in: OCI platform services sit behind LBs you may own; "operationally focused" = healthcheck + failover story is the graded part.

Index

1. Crux & why hard
2. Clarifying questions
3. FR / NFR
4. Back-of-envelope
5. HLD
6. L4 vs L7 — the choice to defend
7. Policies (key code)
8. Healthchecks (key code)
9. Data/config model
10. LB fleet HA
11. Failure modes & ops
12. Trade-offs & defeaters
13. Pop-ups
14. Memorization card

1. Crux & why hard

- Naive: one proxy forwarding to a backend list → it dies (SPOF), it saturates (single NIC), it keeps sending to dead backends.
- What breaks → the minimum primitives: **fleet of stateless LBs** (HA), **balancing policy** (distribute), **healthchecks** (stop sending to the dead), **connection/session handling** (L4 vs L7 decision).
- 🗣️ "I'll design an L7 HTTP load balancer and call out where an L4 answer differs — L4 forwards connections and never parses bytes; L7 terminates, inspects, and routes requests."

2. Clarifying questions (assumed answers)

- L4 or L7? → choose L7 for HTTP APIs (routing, TLS termination, retries); mention L4 for raw TCP/perf
- Internal or edge? → edge: add TLS + WAF-ish concerns; internal: mTLS
- Scale? → assume 100k rps, 500 backends, p99 added latency budget < 5 ms
- Session affinity needed? → support it, default off

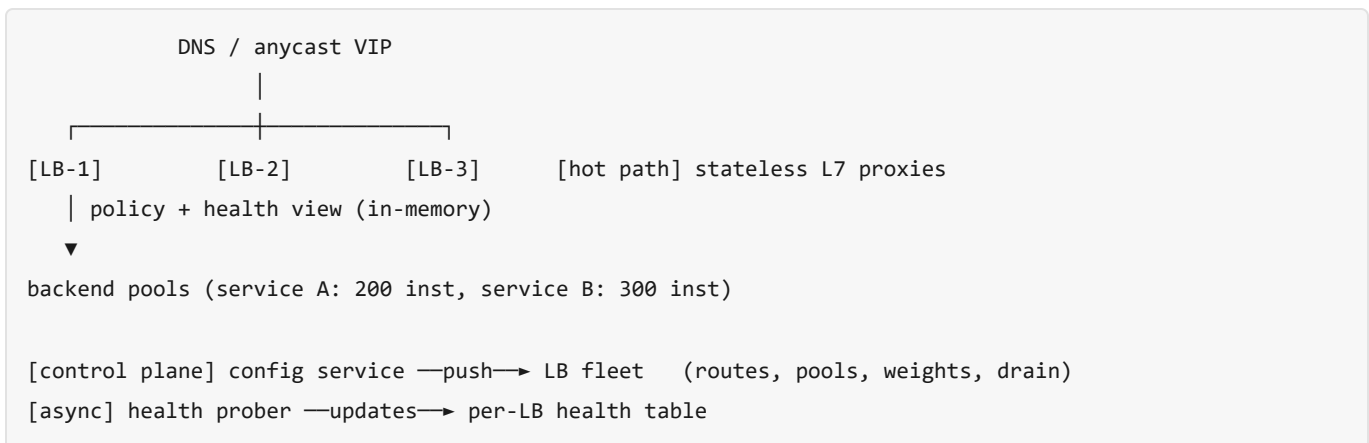
3. FR / NFR

Req	Target	Enforced by
Distribute requests	even under skew	policy (least-conn / P2C)
Stop routing to dead backends	detect < 10 s	active + passive healthchecks
LB itself HA	no SPOF	anycast/ECMP or VIP failover fleet
Added latency	p99 < 5 ms	in-memory routing, keep-alive pools
Config changes	no restart	control plane pushes, versioned config

4. Back-of-envelope

- 100k rps × ~2 KB avg = ~200 MB/s aggregate → a few LB nodes handle it; CPU cost is TLS, not routing — so what: size the fleet by TLS handshakes/s, not rps
- 500 backends × 2 probes/s = 1k probe rps — negligible; per-LB independent probing at 10 LBs = 10k — still fine, but at 10k backends switch to shared health service — so what: healthcheck architecture changes with fleet size

5. HLD



- Walkthrough: client → VIP → one LB (ECMP hash) → LB picks backend by policy among healthy set → proxies request (keep-alive pool) → response; passive health signal recorded from the outcome.

6. L4 vs L7 — defend the choice

	L4	L7
Sees	IP:port, connections	full HTTP request
Can do	NAT/DSR forward, connection balance	path/header routing, TLS termination, retries, rate limit
Cost	~0 parse cost, millions cps	parse + TLS CPU
Use	raw TCP, extreme scale front tier	API traffic (this design)

- 🔗 "Real stacks layer both: L4 (ECMP/anycast) spreads across the L7 fleet; L7 does the smart routing."

7. Policies (key code)

- RR (baseline) · weighted RR (capacity skew) · least-connections (long requests) · **power-of-two-choices** (cheap + near-optimal under skew) · consistent hash (session/cache affinity, sd11 ring)

```
import random

def pick_p2c(backends):
    # backends: [{conns, healthy, weight}]
    healthy = [b for b in backends if b["healthy"]]
    a, b = random.sample(healthy, 2)
    return a if a["conns"] / a["weight"] <= b["conns"] / b["weight"] else b
# O(1); avoids least-conn's "thundering to the emptiest" herd
```

```
Backend pickP2C(List<Backend> healthy, Random rnd) {
    Backend a = healthy.get(rnd.nextInt(healthy.size()));
    Backend b = healthy.get(rnd.nextInt(healthy.size()));
    return (double) a.conns() / a.weight() <= (double) b.conns() / b.weight() ? a : b;
}
// O(1); sample-two beats global-min scan and needs no shared sorted structure
```

8. Healthchecks (key code)

- **Active:** probe `/healthz` every 2 s; K consecutive fails → OUT; M consecutive passes → IN (hysteresis, no flapping)
- **Passive:** real-request failures (5xx/timeout/conn refused) count against the backend — catches what probes miss
- Half-open recovery: readmit with small weight first (slow start) or one canary request

```
class HealthState:
    def __init__(self, fall=3, rise=2):
        self.fall, self.rise = fall, rise
        self.fails = self.passes = 0
        self.healthy = True

    def record(self, ok):
        if ok:
            self.passes += 1
            self.fails = 0
            if not self.healthy and self.passes >= self.rise:
                self.healthy = True
        else:
            self.fails += 1
            self.passes = 0
            if self.healthy and self.fails >= self.fall:
                self.healthy = False
# fall/rise thresholds = the hysteresis that prevents flapping
```

```

class HealthState {
    private final int fall = 3, rise = 2;
    private int fails, passes;
    volatile boolean healthy = true;

    synchronized void record(boolean ok) {
        if (ok) { passes++; fails = 0; if (!healthy && passes >= rise) healthy = true; }
        else    { fails++;  passes = 0; if (healthy && fails >= fall) healthy = false; }
    }
}
// volatile read on hot path; state flips only under lock

```

9. Data/config model

```

-- control-plane store (LBs hold this in memory, versioned)
CREATE TABLE pools (
    pool_id      TEXT PRIMARY KEY,
    policy       TEXT NOT NULL,          -- RR | WRR | LEAST_CONN | P2C | HASH
    hc_path      TEXT NOT NULL DEFAULT '/healthz',
    hc_interval_ms INT NOT NULL DEFAULT 2000
);
CREATE TABLE backends (
    pool_id      TEXT REFERENCES pools,
    addr         TEXT NOT NULL,          -- ip:port
    weight       INT  NOT NULL DEFAULT 1,
    state        TEXT NOT NULL DEFAULT 'IN', -- IN | DRAIN | OUT (admin) ; health is runtime
    PRIMARY KEY (pool_id, addr)
);

```

- DRAIN = stop new, finish in-flight — the deploy story interviewers want to hear.

10. LB fleet HA

- Edge: anycast VIP + ECMP across LBs; per-flow hash keeps a connection on one LB
- Simpler on-prem: active-passive VIP with VRRP/keepalived
- LBs stateless (health + config rebuilt at boot) → static stability: keep routing on last-known config if control plane is down

11. Failure modes & ops

- Backend dies mid-request → passive check + retry idempotent GETs only (? "retry POST?" — only with idempotency key, sd14)
- Zombie backend (passes /healthz, fails real work) → deep healthcheck hits a dependency-touching endpoint; passive checks catch the rest
- Healthcheck storm after config push → jitter probe timers

- Metrics: per-backend rps / p99 / 5xx / OUT-count; alarm on pool healthy-fraction < 50% (that's an us problem, not a backend problem)

12. Trade-offs & defeaters

- Defeater: "LB 单点" — the LB itself must be a fleet behind ECMP/VRRP
- Defeater: "least-conn everywhere" — global least-conn herds to one node on cold start; P2C fixes cheaply
- Defeater: "all backends fail HC at once" → likely HC bug/dependency: fail open (serve last-known-healthy set) rather than route to nothing
- Session affinity vs balance: hash affinity skews load; prefer stateless backends, affinity only for caches

13. Pop-ups

- ? How does the LB scale itself? → stateless fleet + ECMP; TLS offload is the CPU cost, scale by handshake rate
- ? Connection draining on deploy? → DRAIN state, finish in-flight, then OUT
- ? How do retries not amplify an outage? → retry budget (e.g. $\leq 10\%$ of requests), only idempotent methods, jittered backoff
- L5: global LB = anycast + geo-DNS + per-region health published to DNS (low TTL)

14. Memorization card

- L4 forwards connections, L7 routes requests (TLS, paths, retries) — layer both at scale
- Policy: P2C by default; hash for affinity; weights for capacity
- Health: active (fall 3 / rise 2 hysteresis) + passive (real failures); deep check for zombies; fail open if everything looks dead
- HA: anycast/ECMP fleet, stateless, static stability; DRAIN for deploys